Data Structures
Assignment – 0 2

By
Sharjeel
L1F24BSCS0506
Instructor: Sammra Habib

Section A – Problem Understanding

This assignment involves implementing three data structures in C++: Queue, Circular Queue, and Singly Linked List. The objective is to apply these structures in simple smart campus management systems while implementing everything manually without using STL libraries.

In this project, three modules were developed:

- **Service Desk Token Manager** using Queue
- **Lab Task Scheduler** using Circular Queue
- **Student Help Request Tracker** using Singly Linked List

The first module, Smart Campus Service Desk Token Manager, uses a Queue to manage students in a FIFO (First-In-First-Out) order. Each student has a Token ID, Name, and Service Type. Basic operations such as enqueue, dequeue, display, and checking the front/empty state are implemented. This module demonstrates why Queue is suitable for fair service systems, while Stack is not. The starting token number is based on the student's roll number.

The second module, Lab Computer Task Scheduler, is implemented using a Circular Queue with fixed capacity. It includes Task ID, Task Name, and Estimated Time. Operations such as enqueue, dequeue, display, and overflow/underflow detection are included. This module shows how a Circular Queue efficiently reuses empty spaces, unlike a linear queue that may waste memory due to false overflow. Capacity is also personalized.

The third module, Student Help Request Tracker, uses a Singly Linked List to manage dynamic requests. Each node contains Request ID, Student Name, Issue Type, and Status. It supports insertion, deletion, searching, and updating records. This structure is suitable for dynamic data due to

flexible memory allocation and efficient pointer-based operations. Overall, the assignment focuses on understanding and applying linear data structures in real-world scenarios while ensuring manual implementation and conceptual clarity.

Additionally, AI tools were used as learning assistants for planning, debugging, and improvement. However, all AI-generated suggestions were reviewed, modified, and tested manually to ensure correctness and understanding.

Section B – Design Logic and Data Structure Justification

The design of this assignment is based on selecting the most appropriate data structure for each real-world scenario to ensure efficiency and correctness. Each module is implemented using a custom class-based approach in C++ with manual memory and pointer handling, without relying on STL containers.

For the **Service Desk Token Manager**, a Queue is used because it follows the **FIFO (First-In-First-Out)** principle. This ensures that students are served in the same order in which they arrive, making the system fair and organized. The class design includes a dynamic structure to store Token ID, Student Name, and Service Type, with operations such as enqueue, dequeue, and display.

For the **Lab Task Scheduler**, a Circular Queue is implemented to efficiently utilize fixed memory space. Unlike a linear queue, which can cause false overflow due to unused spaces, the circular approach reuses deleted positions by connecting the rear back to the front. This improves memory efficiency while maintaining **O(1)** complexity for enqueue and dequeue operations. The design also includes front and rear tracking for proper circular behavior.

For the **Student Help Request Tracker**, a Singly Linked List is used due to its dynamic memory allocation capability. It allows flexible insertion and deletion without shifting elements, making it suitable for continuously changing data. Pointer manipulation is used for node insertion, deletion, and traversal. Search operations run in **O(n)** time due to sequential access.

Overall, each data structure was selected based on its suitability to the problem requirements, focusing on efficiency, memory usage, and real-world application behavior.

Section C – AI Usage Record

AI tool (ChatGPT) was used as a learning assistant during the development of this assignment. The purpose of using AI was to understand requirements, plan class structures, and improve debugging and design decisions. All AI responses were carefully reviewed, modified, and implemented manually to ensure correctness and conceptual understanding.

Link: https://chatgpt.com/share/69d96801-f484-83e8-b16b-2e3de3b24e24

Prompts

Prompt – 01: This document is my data structures assignment 02. Explain briefly in bullet points what do I have to do in this assignment. Don't start any coding, just let me understand it first so I can understand what the instructor wants in bullet points especially about ai usage documentation part so I don't miss anything here.

Prompt – 02: For task 01, I have already implemented generic template based linear queue with polymorphism. What I need you to do make a class of queue **StudentTokenManager** and use my custom class queue in it. "The starting token number must be based on the student's roll number" -> My roll no. is L1F24BSCS0506.

Prompt – 03: Now for task 03, i want you to do the same as previously. I have made my own custom circular queue class use that and make a class **LabTaskScheduler**.

Prompt – 04: Now for task 03, I want you to implement class **RequestTracker** using my custom singly linked list implementation

Summary of AI Contribution

- Helped in understanding requirements and structuring logic
- Assisted in class design and integration of custom data structures
- Supported debugging and conceptual clarity

Limitations Observed in AI Output

- Suggested incorrect or unsupported direct attribute access
- Missed operator overloading requirements in custom classes
- Did not fully follow abstraction rules of implemented structures

Section D – Testing and Output

Task – 01

AI mistake Logs:

- Ai also violated the abstraction rules, my currSize attribute is protected but in its implementation, it tried to get currSize just by writing .currSize which will obviously give error, so I added a custom getter function getCurrSize() to remove this mistake.
- It also didn't do operator overloading for student structure that caused error and I had to correct it.
- It also wrongfully tried to use [] operator in this class which my custom queue class didn't support before. So I had to implement new method `T& operator[](int);`

New Implementation:

I added a new functionality search a student by his/her name and token id.

Stack VS Queue

A **Queue (FIFO)** is suitable for a service desk because it serves students in the order they arrive, ensuring fairness and proper token management.

A **Stack (LIFO)** is not suitable because it serves the last arriving student first, which is unfair in a queue-based system like a service desk.

Output Screenshot

```
sharjeel < task-01 >: run student-token-manager
Token Issued: 506
Token Issued: 507
Token Issued: 508
Token Issued: 509
Token Issued: 510
Token Issued: 511


--- Current Queue ---
Token: 506 | Name: Alice | Service: Fee Payment Issue
Token: 507 | Name: Bob | Service: Library Fine Issue
Token: 508 | Name: Charlie | Service: IT Support Request
Token: 509 | Name: David | Service: Transcript Issue
Token: 510 | Name: Emma | Service: Exam Registration Issue
Token: 511 | Name: Frank | Service: Transport Card Activation Issue

Next Token: 506 | Name: Alice | Service: Fee Payment Issue


Student Found:
Token ID: 508
Name: Charlie
Service Type: IT Support Request

Serving Token: 506 | Name: Alice | Service: Fee Payment Issue
Serving Token: 507 | Name: Bob | Service: Library Fine Issue

After serving two students:

--- Current Queue ---
Token: 508 | Name: Charlie | Service: IT Support Request
Token: 509 | Name: David | Service: Transcript Issue
Token: 510 | Name: Emma | Service: Exam Registration Issue
Token: 511 | Name: Frank | Service: Transport Card Activation Issue

Total Waiting Students: 4
sharjeel < task-01 >: |
```

Task – 02

AI mistake Logs

- AI seems to despise operator overloading. It doesn't creating methods for operator overloading to properly handle Task structure. I had to implement on my own the new method
  ```
  ostream& operator<<(ostream&, const Task&);
  ```

Linear VS Circular

A simple linear queue wastes space because after several dequeue operations, the front part becomes empty but cannot be reused. As a result, even if there is free space, new elements cannot be inserted once the rear reaches the end of the array. This leads to "false overflow."

A circular queue solves this by connecting the end back to the beginning, allowing reuse of empty spaces efficiently.

Output Screenshot

```
sharjeel < task-02 >: run lab-task-scheduler
Task Added: 506 | Run Compiler Test | 15 min
Task Added: 507 | Execute Sorting Program | 20 min
Task Added: 508 | Debug Queue Implementation | 25 min
Task Added: 509 | Compile Lab Report Code | 10 min
Task Added: 510 | Memory Leak Analysis | 30 min


--- Lab Task Queue ---
Current Size: 5
Max Size: 11
Front: 0
Rear: 5
0. ID: 506 | Name: Run Compiler Test | Time: 15 min
1. ID: 507 | Name: Execute Sorting Program | Time: 20 min
2. ID: 508 | Name: Debug Queue Implementation | Time: 25 min
3. ID: 509 | Name: Compile Lab Report Code | Time: 10 min
4. ID: 510 | Name: Memory Leak Analysis | Time: 30 min

Next Task -> 506 | Run Compiler Test | 15 min

Front: 0
Rear: 5

Completed Task:
ID: 506 | Name: Run Compiler Test | Time: 15 min

--- Lab Task Queue ---
Current Size: 4
Max Size: 11
Front: 1
Rear: 5
1. ID: 507 | Name: Execute Sorting Program | Time: 20 min
2. ID: 508 | Name: Debug Queue Implementation | Time: 25 min
3. ID: 509 | Name: Compile Lab Report Code | Time: 10 min
4. ID: 510 | Name: Memory Leak Analysis | Time: 30 min
Completed Task:
ID: 507 | Name: Execute Sorting Program | Time: 20 min

--- Lab Task Queue ---
Current Size: 3
Max Size: 11
Front: 2
Rear: 5
2. ID: 508 | Name: Debug Queue Implementation | Time: 25 min
3. ID: 509 | Name: Compile Lab Report Code | Time: 10 min
4. ID: 510 | Name: Memory Leak Analysis | Time: 30 min
Completed Task:
ID: 508 | Name: Debug Queue Implementation | Time: 25 min
```

```
--- Lab Task Queue ---
Current Size: 2
Max Size: 11
Front: 3
Rear: 5
3. ID: 509 | Name: Compile Lab Report Code | Time: 10 min
4. ID: 510 | Name: Memory Leak Analysis | Time: 30 min
Completed Task:
ID: 509 | Name: Compile Lab Report Code | Time: 10 min

--- Lab Task Queue ---
Current Size: 1
Max Size: 11
Front: 4
Rear: 5
4. ID: 510 | Name: Memory Leak Analysis | Time: 30 min
Completed Task:
ID: 510 | Name: Memory Leak Analysis | Time: 30 min
No tasks in scheduler.

Now after emptying all queue we can again add values in queue proving that empty spaces are reused

Front: 5
Rear: 5
Task Added: 511 | Fix System Crash Issue (Custom Debug Task) | 40 min
Task Added: 512 | Run printer | 27 min

New Task successfully added:

--- Lab Task Queue ---
Current Size: 2
Max Size: 11
Front: 5
Rear: 7
5. ID: 511 | Name: Fix System Crash Issue (Custom Debug Task) | Time: 40 min
6. ID: 512 | Name: Run printer | Time: 27 min

Next Task -> 511 | Fix System Crash Issue (Custom Debug Task) | 40 min
sharjeel < task-02 >:
```

Task – 03

Ai mistakes logs

- Ai seems to be hallucinating at this point. It was again and again missing operator overloading methods for '==' and '<<' operator, so again I had to implement those methods on my own.

LinkedList VS Array

A linked list is suitable for this student request tracker because because the number of requests is dynamic and continuously changing. Unlike a fixed array, it does not require a predefined size and can grow or shrink as requests are inserted or deleted . This avoids memory wastage and overflow issues that occur in arrays when capacity is exceeded. It also allows efficient insertions at the beginning (urgent requests) and flexible deletions without shifting elements.

In contrast, arrays are fixed in size and may cause overflow or unused space. Pointer movement allows nodes to be connected or removed by updating links instead of shifting data.

Pointer Movement During Insertion and Deletion

During insertion, a new node is created and its next pointer is adjusted to link it either to the current head (urgent request) or to the last node in the list (normal request using tail insertion). During deletion, pointers are moved node by node using a temporary pointer until the target Request ID is found. Once found, the previous node's next pointer is updated to skip the deleted node, and memory is effectively unlinked from the list.

Output Screenshot

```
sharjeel < task-03 >: run help-request-tracker
Request Added: ID 506
Request Added: ID 507
Request Added: ID 508
Request Added: ID 509
Request Added: ID 510
URGENT Request Added: ID 511

--- All Requests ---
0. ID: 511 | Name: Bilal | Issue: Exam Roll Number Issue | Status: Urgent
1. ID: 506 | Name: Ali | Issue: Library Card Issue | Status: Pending
2. ID: 507 | Name: Sara | Issue: Course Registration Error | Status: Pending
3. ID: 508 | Name: Usman | Issue: WiFi Connectivity Issue | Status: Pending
4. ID: 509 | Name: Ayesha | Issue: Fee Voucher Problem | Status: Pending
5. ID: 510 | Name: Hassan | Issue: Hostel Electricity Issue | Status: Pending
Status updated for ID 508

After Update:

--- All Requests ---
0. ID: 511 | Name: Bilal | Issue: Exam Roll Number Issue | Status: Urgent
1. ID: 506 | Name: Ali | Issue: Library Card Issue | Status: Pending
2. ID: 507 | Name: Sara | Issue: Course Registration Error | Status: Pending
3. ID: 508 | Name: Usman | Issue: WiFi Connectivity Issue | Status: Resolved
4. ID: 509 | Name: Ayesha | Issue: Fee Voucher Problem | Status: Pending
5. ID: 510 | Name: Hassan | Issue: Hostel Electricity Issue | Status: Pending
sharjeel < task-03 >:
```

Section E – Reflection & Learning Summary

Throughout this assignment, AI was used as a **support tool for understanding and structuring ideas**, not as a replacement for coding.

What AI helped with:

- Explaining data structure logic
- Structuring classes and modules
- Suggesting design improvements
- Debugging conceptual errors

What AI did NOT do well:

- Often missed operator overloading requirements
- Suggested unsafe or direct attribute access
- Sometimes ignored custom implementation constraints
- Did not fully respect abstraction rules in custom classes

Overall, this assignment improved my understanding of linear data structures, enhanced my debugging and design skills, and demonstrated how AI can be effectively used as a learning assistant when combined with proper validation and independent thinking.